



UNIVERSAL ROBOTS

Script Library Tutorial

Original Instructions (EN)

Version 1.0

By Phillip Grambo | pgra@universal-robots.com

The information contained herein is the property of Universal Robots A/S and shall not be reproduced in whole or in part without prior written approval of Universal Robots A/S.

The information herein is subject to change without notice and should not be construed as a commitment by Universal Robots A/S. This manual is periodically reviewed and revised.

Universal Robots A/S assumes no responsibility for any errors or omissions in this document.

Copyright © 2024 by Universal Robots A/S
The Universal Robots logo is a registered trademark of Universal Robots A/S.

1 PURPOSE

URScript is the underlying code that makes the robot act. Users can create extensive additional functionality by using URScript. This guide is a walkthrough of how to develop that functionality, and examples of what can be made.

2 TABLE OF CONTENTS

1	Purpose	3
3	What is this guide?	4
4	What is URScript?	5
5	URScript Manual	8
6	Tools for Writing URScript	9
6.1	On the Teach Pendant	9
6.2	URSim	10
6.3	Notepad	10
6.4	Notepad++	11
6.5	VS Code	12
6.6	Document Writers	14
6.7	VNC	14
6.8	FTP	14
7	Making a Simple Function with def():	16
8	Inserting a .script File Into a Before Start Sequence	17
9	Functions That Call Other Functions	25
10	TCP/IP Socket Communication Example	27
11	Global vs Local Variables	29
12	Writing an Entire Program in URScript	32
13	Organizing and Managing Your Library	35
14	Change Log	37

3 WHAT IS THIS GUIDE?

This guide is a walkthrough of how to make your own script library for a UR robot. A script library is a collection of user defined functions that build upon the existing functions native to the environment. These user-defined functions are useful for making reusable blocks of code to provide new, different, or complicated functionality to the robot.

This guide is not a master reference for URScript programming in general. That document, called the UR Script Manual, can be found on our website at: <https://www.universal-robots.com/download/manuals-e-series/script/script-manual-e-series-sw-512/>.

This guide is also not a “What is programming?” instructional document. While this guide does offer a brief description of what programming is, this is meant as a refresher. This guide assumes a rudimentary knowledge of what programming is, and why you might program something.

- Background Concepts: What is programming?
- Programming is the act of making instructions for a computer to follow.
- Code is actual instructions for the computer.
- Code must be written in a Programming Language that the computer can read, like Python, C, C++, Java, or URScript. Computers have to read these exact and explicit languages, because human languages like English are too vague and imprecise for consistent results.
- Code is often written in a human-accessible software like PolyScope, and then Compiled into a more detailed language like URScript, which the computer will then use.
- Computers will do exactly what you have told them to do, regardless of what you actually meant for them to do.

4 WHAT IS URSRIPT?

URScript is a custom programming language developed by Universal Robots. It is similar to Python, but not exactly the same. It does not have the entire capacity of Python, but it has added functionality specific to our robot and controller. URScript files are saved with a .script file name.

When you press Play on the teach pendant, a .script file is compiled from the program file, installation file, and any installed URCaps. The recently generated .script file, which has the same name as the .urp file, will overwrite an existing .script file if it has the same name. The .script file will then be sent to the controller, which executes the .script file, causing the robot to move and act as described in the program file.

PolyScope is a GUI (Graphical User Interface) that users control to put premade program nodes (Waypoints, Set nodes, If statements, URCap commands, etc.) in order. These nodes compile very reliably into working .script files. Polyscope also allows users to insert script nodes into the program, which will then contain either a single line or a file of URScript commands. This guide will walk you through using and inserting you own .script files.

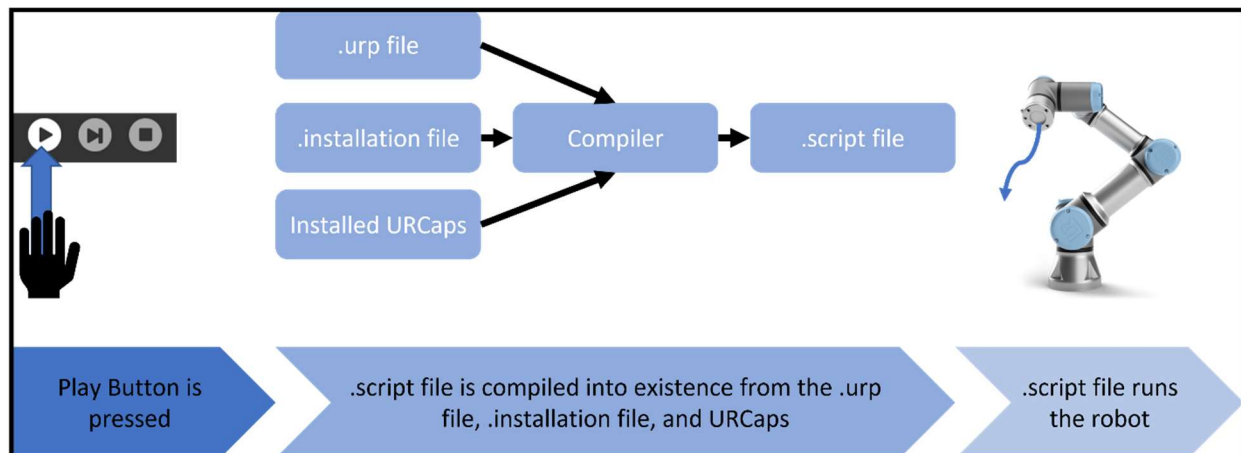


Figure 1: Diagram of .script File Creation when Play Button is Pressed. Note that the .script File is Not Compiled Until the Program is Played, and Is Not Recompiled Until the Program is Changed and Played Again.

```

1 def prog_to_script():
2   set_standard_analog_input_domain(0, 1)
3   set_standard_analog_input_domain(1, 1)
4   set_tool_analog_input_domain(0, 1)
5   set_tool_analog_input_domain(1, 1)
6   set_analog_outputdomain(0, 0)
7   set_analog_outputdomain(1, 0)
8   set_tool_voltage(0)
9   set_input_actions_to_default()
10  set_tcp(p[0.0,0.0,0.1,0.0,0.0,0.0])
11  set_payload(0.0)
12  set_gravity([0.0, 0.0, 9.82])
13  global Punkt_1=p[-0.137,-0.429,-0.0478,-0.001,3.116,0.038]
14  while (True):
15    $ 1 "Robot Program"
16    $ 2 "MoveJ"
17    $ 3 "Waypoint_1"
18    movej([-1.876, -2.277, -1.000, -1.465, 1.587, -0.306], a=1.396, v=1.047)
19    $ 4 "Waypoint_2"
20    movej([-1.966, -2.074, -1.755, -0.915, 1.584, -0.396], a=1.396, v=1.047)
21    $ 5 "Wait: 5.0"
22    sleep(5.0)
23  end
24 end

```

Figure 2: A Simple Robot Program that Moves from Waypoint_1 to Waypoint_2, then waits for 5 seconds, in its Compiled FileName.script Format that Executes on the UR Controller. Lines 2-12 are from the .installation File. Program Node Text is Shown as: \$ LineNumber "Node Text".

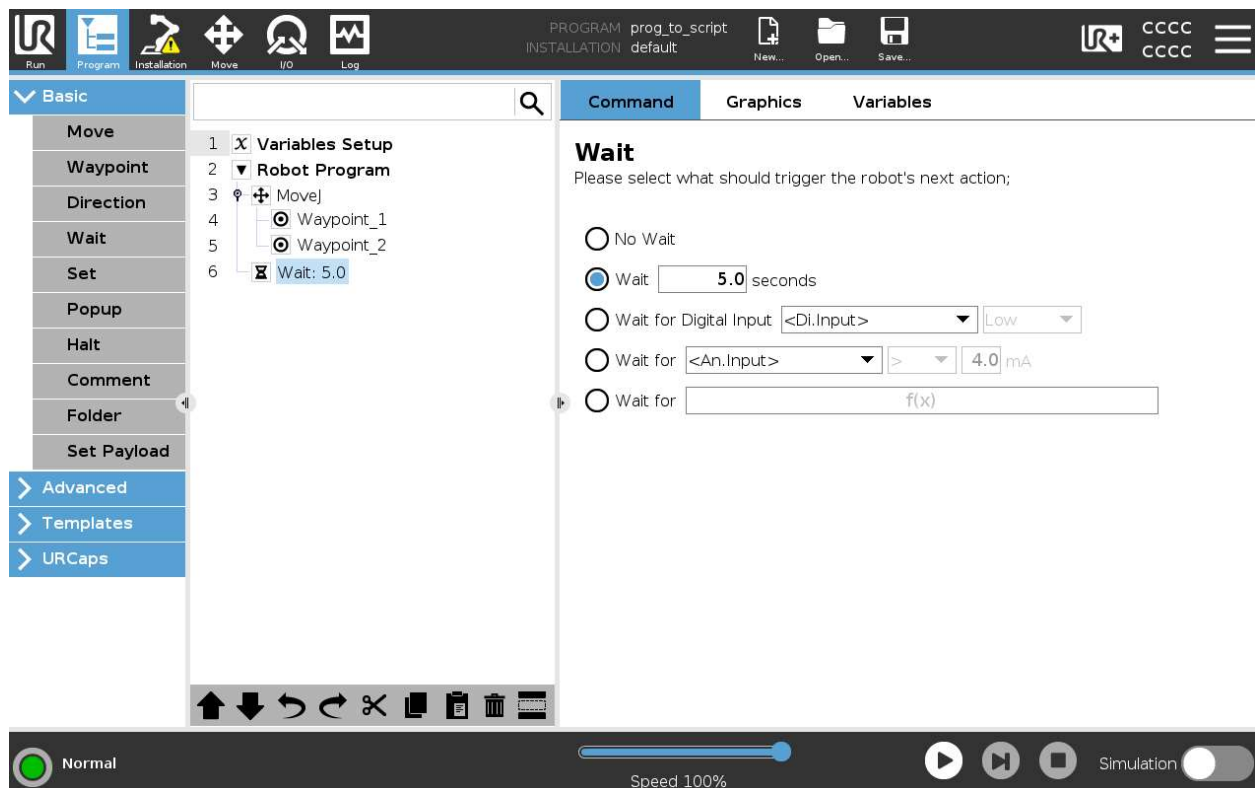


Figure 3: The PolyScope Program that was Compiled into the Script File in the Previous Image



```
15  set_tool_digital_output_mode(0, 1)
16  set_tool_digital_output_mode(1, 1)
17  set_tool_voltage(0)
18  set_target_payload(0.000000, [0.000000, 0.000000, 0.000000]),
19  # begin: URCap Installation Node
20  #   Source: Robotiq_Grippers, 1.8.11.21258, Robotiq Inc.
21  #   Type: Vacuum
22  #####
23  # Vacuum Grip Check
24  #####
25
26  vacuumGripCheckThread1 = 0
27  vacuumGripCheckThread2 = 0
28  vacuumGripCheckThread3 = 0
29  vacuumGripCheckThread4 = 0
30  vacuumGripCheckThread1Running = False
31  vacuumGripCheckThread2Running = False
32  vacuumGripCheckThread3Running = False
33  vacuumGripCheckThread4Running = False
34  vacuumGripCheckSocketId = "1"
35  vacuumGripCheckThreadStarted = False
36
37  thread vacuumGripCheck():
38      gripper_socket = vacuumGripCheckSocketId
```

Figure 4: Example `program_name.script` Contents from a URCap

5 URSCRIPT MANUAL

The Universal Robots script manual contains:

- The available variable types
- Flow of control
- How functions work
- Scoping rules
- How URScript interacts with the actual robot controller
- A listing of a known functions
- Other supporting information

The script manual can be downloaded at: <https://www.universal-robots.com/download/manuals-e-series/script/script-manual-e-series-sw-512/>

It can also be read online at: https://myur.universal-robots.com/manuals/content/SW_5_13/Documentation%20Menu/Script%20Manual

Have the URScript manual available while reading and using this guide. The manual is the master reference for what URScript is and how it works. This guide builds on top of that manual, describing a particular use case for URScript.

6 TOOLS FOR WRITING URSCRIPT

A .script file can be written and edited in any text editor. A .script file is just a text file that has been saved with a .script file name.

6.1 ON THE TEACH PENDANT

The Script node insertion can be found on under the Advanced group of nodes. From here, you can type out individual lines, or use the “file” option. In the file editor, you can open existing .script files, or write them from scratch. This is useful for viewing and debugging files on a real robot in the field, but large writing and programming tasks on a touchscreen keyboard is rarely preferred over any other option.

You can plug a USB keyboard into the USB ports on the teach pendant or in the controller to make things easier. However, you may need to change the keyboard to match your locality. USB keyboards will be mapped as Danish layout by default. Letters and numbers will likely be correct, but some special characters may be incorrect. The guide linked below explains how to change the keyboard mapping of USB keyboards.

<https://www.universal-robots.com/articles/ur/application-installation/changing-the-console-keyboard-layout/>

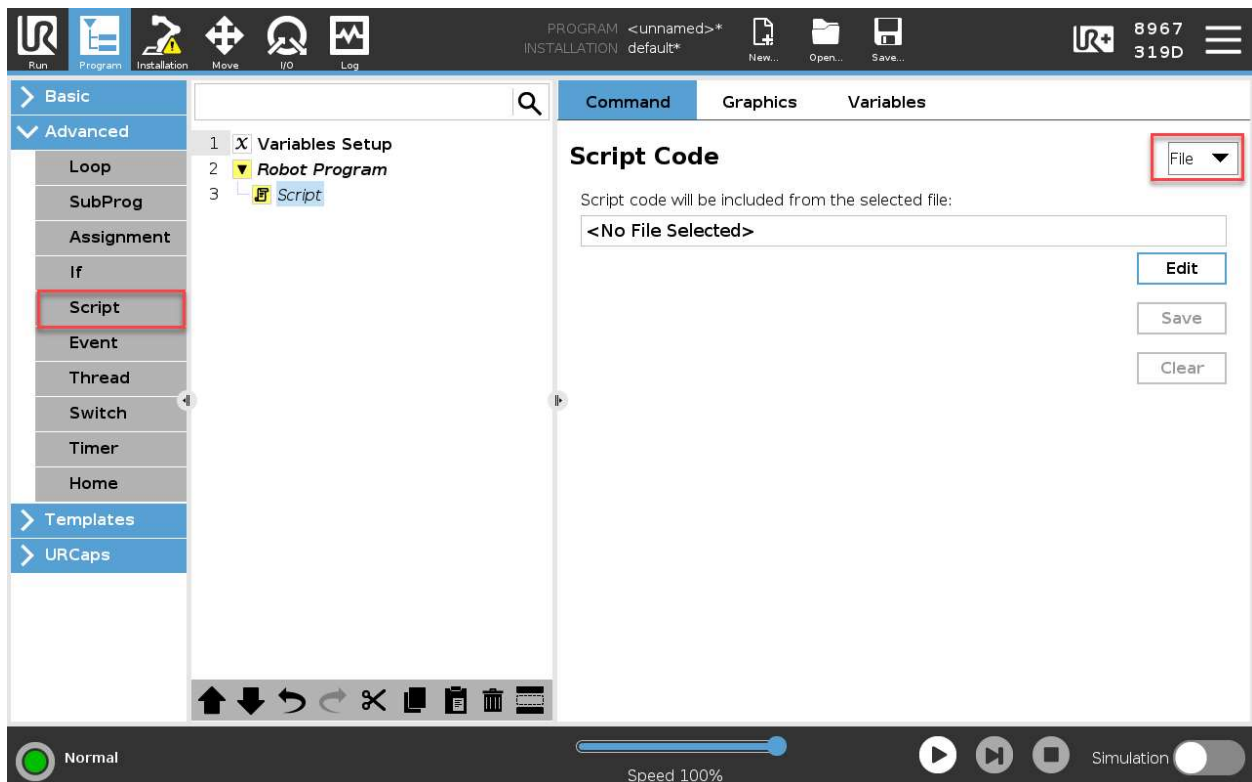


Figure 5: Script Node Insertion in PolyScope

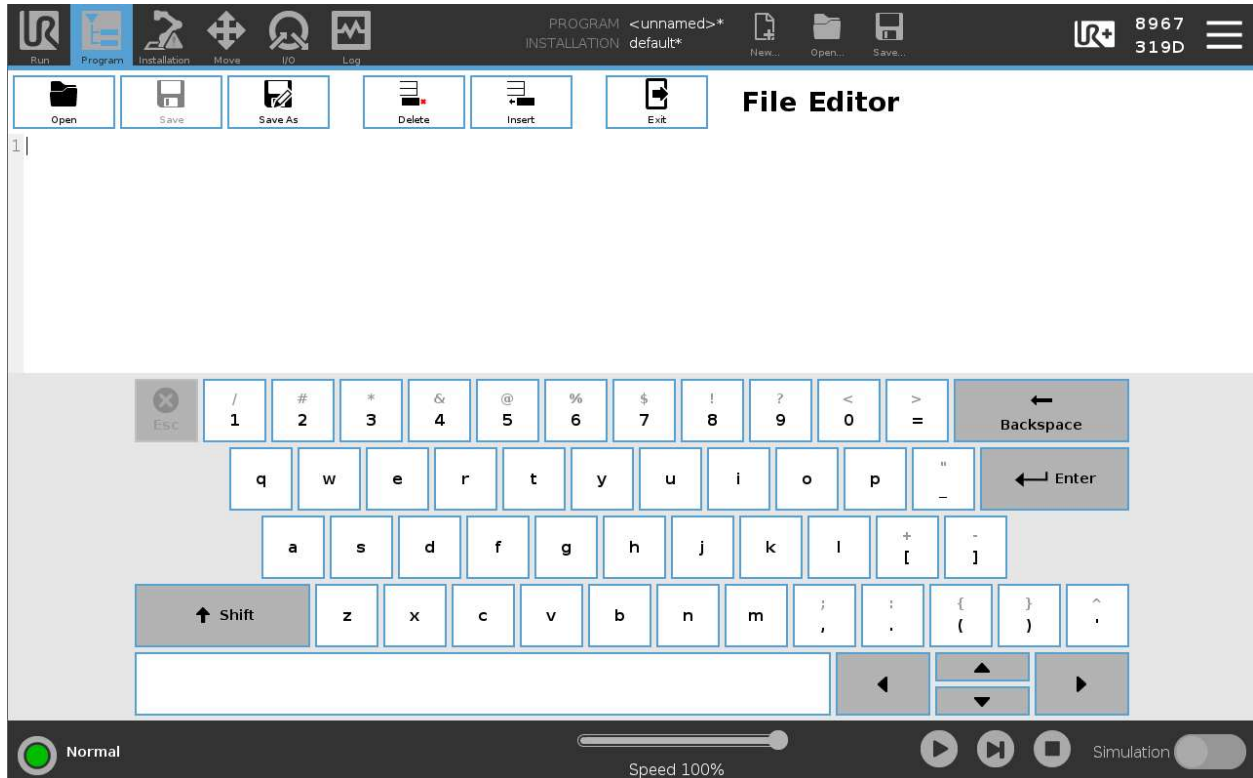


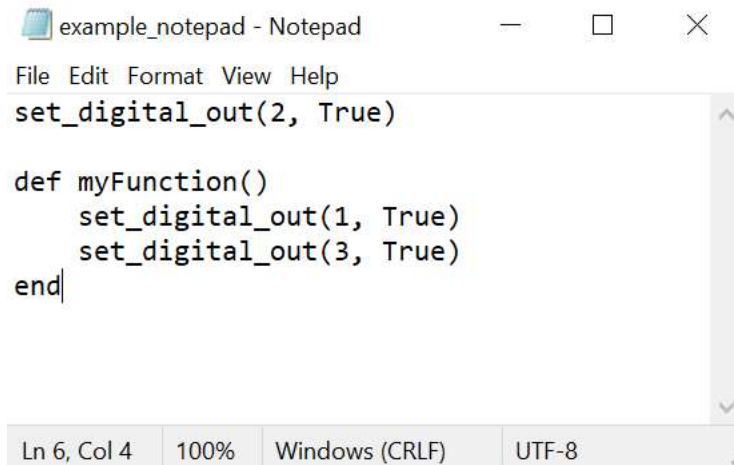
Figure 6: Script File Editor Screen

6.2 URSIM

The offline URSim offers the same programming interface as the teach pendant. Other than being on your computer, it is no different. However, you do have a keyboard and mouse built in, making things easier. Also, you can easily transfer script files from the simulator to repositories, flash drives, emails, and file folder locations, which is very helpful during development and project management tasks.

6.3 NOTEPAD

Basic text editors like Notepad can be used, and most people already have Notepad or a similar software already installed. It lacks several features, like line numbers and color formatting, so it typically is used to inspect and read .script files, but not write or edit them.



```
example_notepad - Notepad
File Edit Format View Help
set_digital_out(2, True)

def myFunction()
    set_digital_out(1, True)
    set_digital_out(3, True)
end

Ln 6, Col 4  100%  Windows (CRLF)  UTF-8
```

Figure 7: Example URScript Code in Notepad

6.4 NOTEPAD++

Notepad++ is a popular free text editor that has some features useful for writing .script files. It can show line numbers, and format by programming language. Setting the language to Python (URScript is most similar to Python) makes for a decent URScript IDE.

<https://notepad-plus-plus.org/>

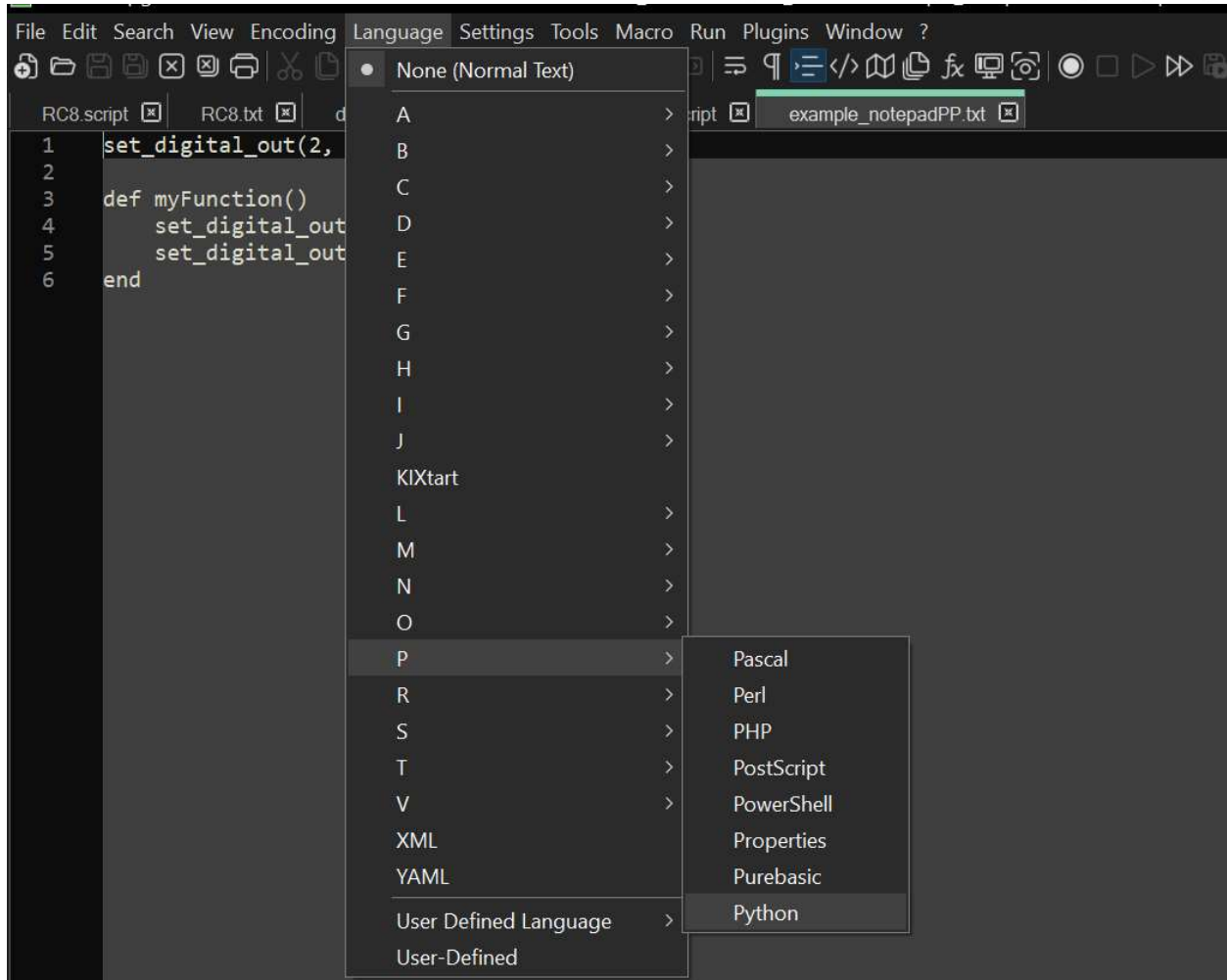


Figure 8: Setting Notepad++ to Python

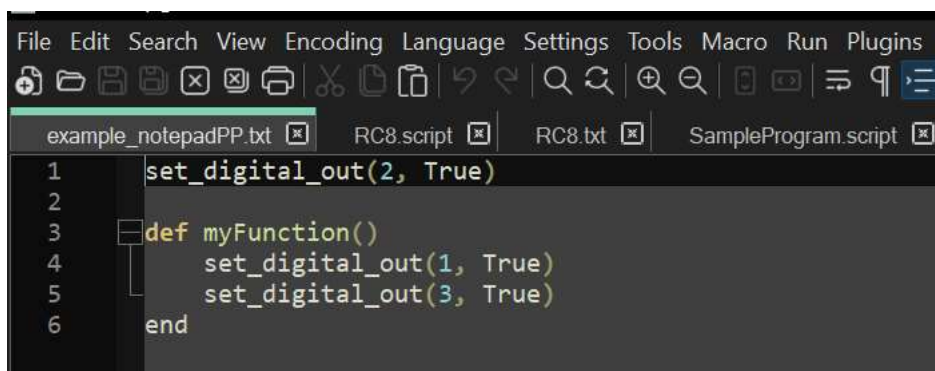


Figure 9: A .script File Notepad++ with Python Set as the Language

6.5 VS CODE

VS Code is a free software IDE from Microsoft. It can be downloaded at <https://code.visualstudio.com/>.

What sets VS Code above the other options is the community developed extension for URScript:

<https://marketplace.visualstudio.com/items?itemName=ahern.urscript>. This extension provides formatting help, function autocompletion, syntax help, and information about function arguments for known functions in the script manual.

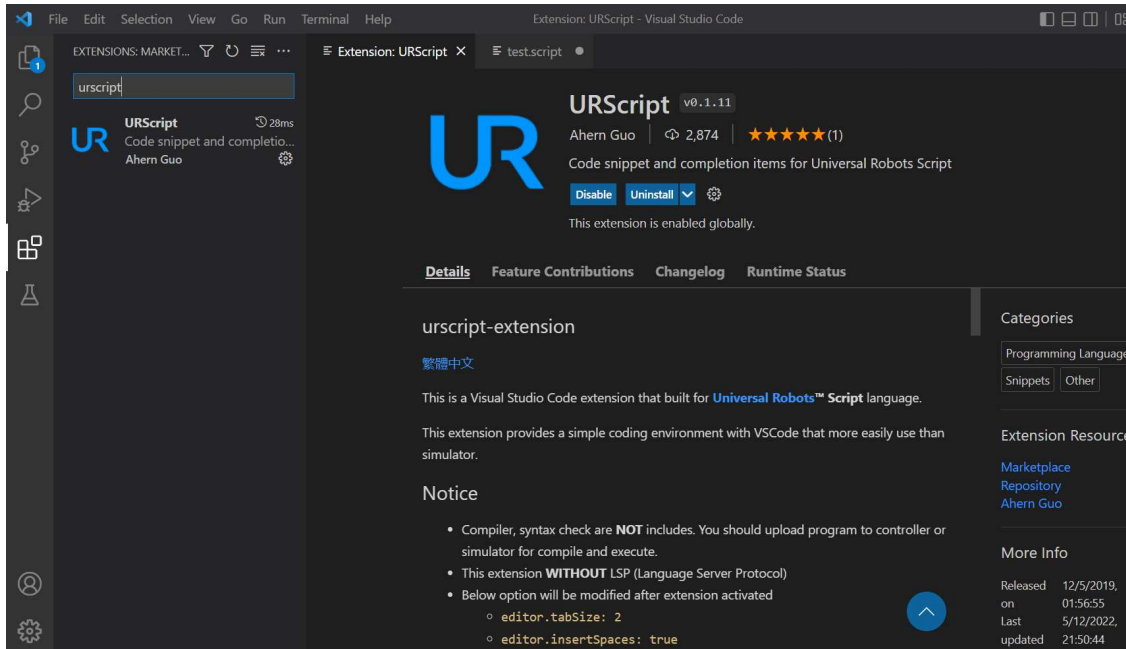


Figure 10: URScript Extension for VS Code

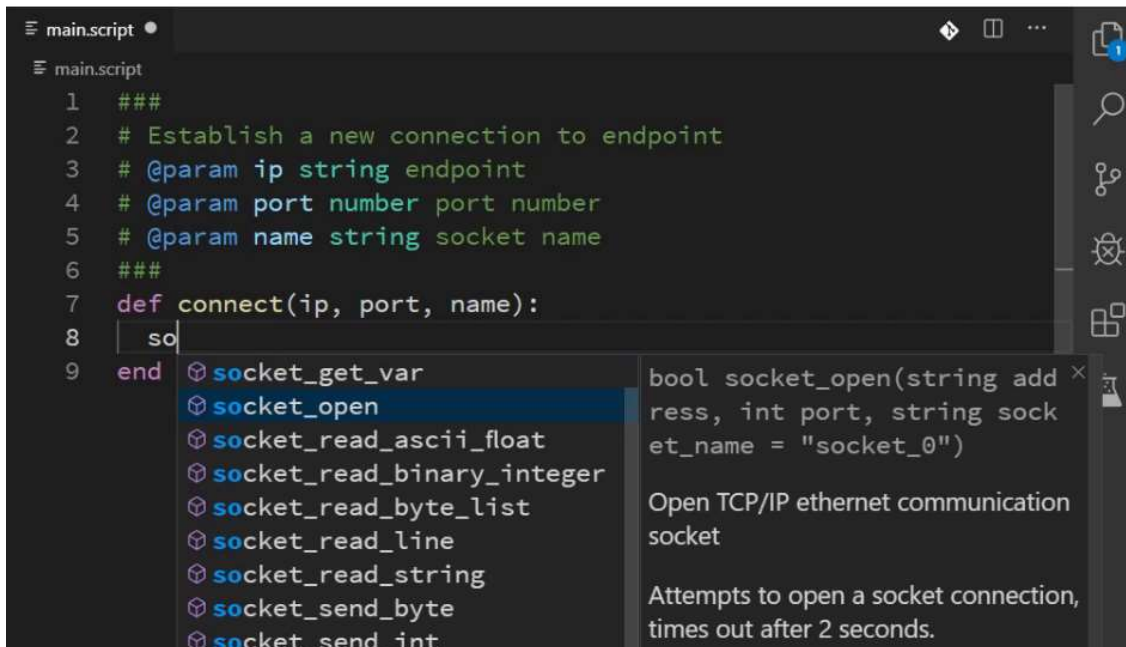


Figure 11: Autocomplete Example for VS Code URScript Extension

Because this extension is developed outside of UR, we cannot guarantee that it is kept up to date.

6.6 DOCUMENT WRITERS

It is possible, but not recommended, to write .script files in a document writing software like MS Word, as these typically try to auto-format and spell-check text to human languages like English, as opposed to a programming language.

6.7 VNC

Before discussing the capabilities of VNC, safety information and disclaimers must be addressed first. Using a VNC connection to our robots violates Single Point of Control, therefore it should not be used in production. It can be used for development and troubleshooting purposes if appropriate precautions are taken. Single Point of Control is a machine safety principle, stating that there can only ever be one method of turning on a machine at any given time. This is crucial for lock-out/tag-out and other safety procedures.

VNC (Virtual Network Computing) is a way of remotely accessing and controlling one computer from another computer. Our robots are run from a computer, so they can be controlled in this manner from a separate computer. Remoting into a robot from across the room or building can be extremely useful for quick edits.

To use VNC, you have to install a VNC server on the robot. The community developed magic file for this is linked below. When you are done using VNC on your robot, it is recommended to either uninstall the server, or disable remote access to the robot in the security settings.

<https://forum.universal-robots.com/t/remote-desktop-to-the-controller-ui/3826/>

6.8 FTP

While FTP is not a writing tool, it is a very useful way to transfer script files from a computer to the robot. The typical workflow for moving files to and from a robot is a flash drive, but FTP allows you to transfer files over an ethernet cable. FTP (File Transfer Protocol) is a way of accessing a computer's file system remotely. To use FTP, you will need software that can act as an FTP Client. UR typically suggests FileZilla (filezilla-project.org) as a free FTP Client. To connect to the robot, you will need the following parameters:

- Host: IP address of the robot
- Username: root
- Password: easybot
- Port: 22

When using FTP, be aware that by inputting a root password, there are no read/write restrictions. If you edit anything outside of the Programs folder, you will likely negatively affect the robot.

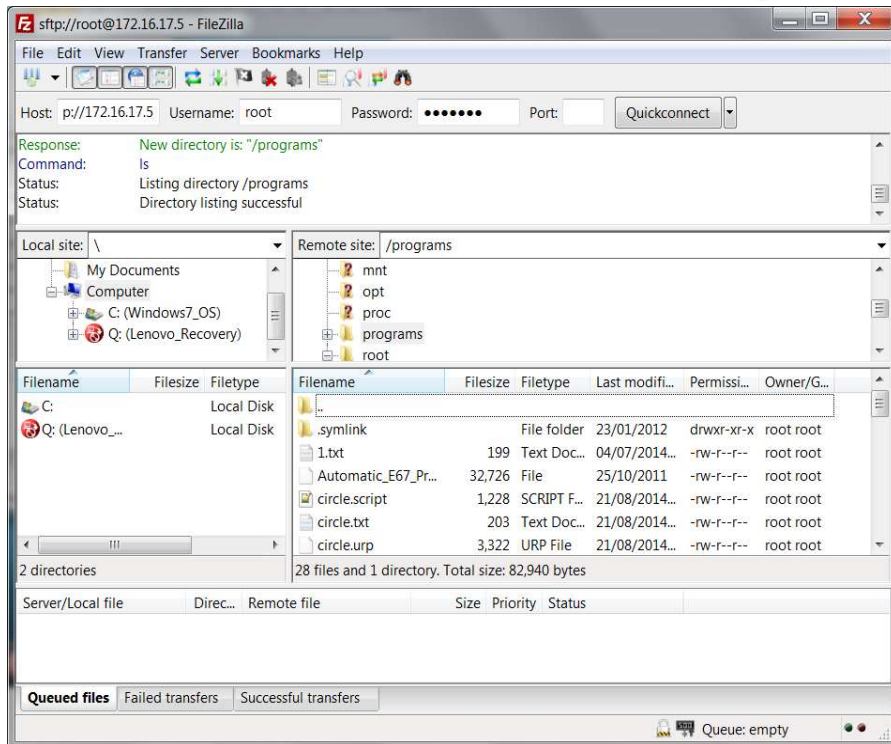


Figure 12: FileZilla Client Connecting a PC to a UR Robot

7 MAKING A SIMPLE FUNCTION WITH DEF():

User defined functions are declared with the `def():` command. All functions must be concluded with `end`. Within the parentheses of the `def():` declaration, comma separated arguments can be passed into the function. Arguments can have default values assigned to them. The parentheses must be followed by a colon. A user defined function can optionally return values. Within the script function, any existing script functions from the script manual or other previously defined user functions can be used. Once a function has been defined, it can be called from within the file, or anywhere else in the PolyScope program.

```
1  def EX_function(optional, arguments, default_value=0):
2
3      # code goes here
4
5      return x # functions can return parameters
6
7  end
8
```

Figure 13: Example .script Function

For a simple example of a user-defined script function that doesn't return a value, let's make a function that turns off the digital outputs 4 through 7. This function could be used upon program start to reset all the outputs connected to a stack light. We will define a function following the structure of the previous figure, and within the program we will use the function `set_digital_out()` from the URScript manual.

```
1  def stack_light_off():
2      set_digital_out(4, False)
3      set_digital_out(5, False)
4      set_digital_out(6, False)
5      set_digital_out(7, False)
6  end
7
```

Figure 14: A User Defined .script Function That Turns Off Outputs 4 Through 7

8 INSERTING A .SCRIPT FILE INTO A BEFORE START SEQUENCE

Now that a .script file has been written, it needs to be inserted into robot program so it can be used.

First, it must be transferred from where it was written, onto the robot. Copy and Paste via a flash drive is likely the easiest. FTP software like FileZilla can also work well.

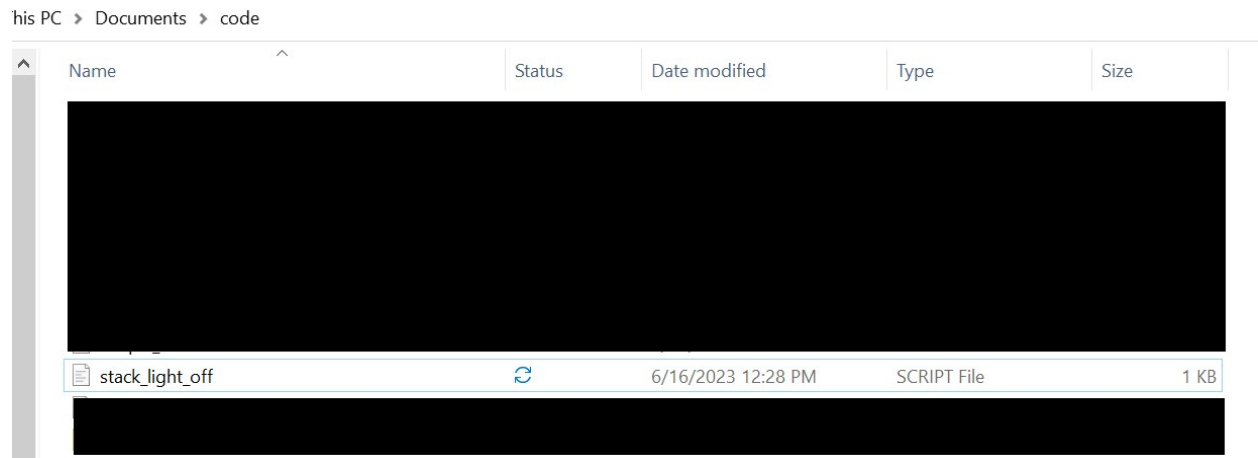


Figure 15: The .script File in a Windows File Folder

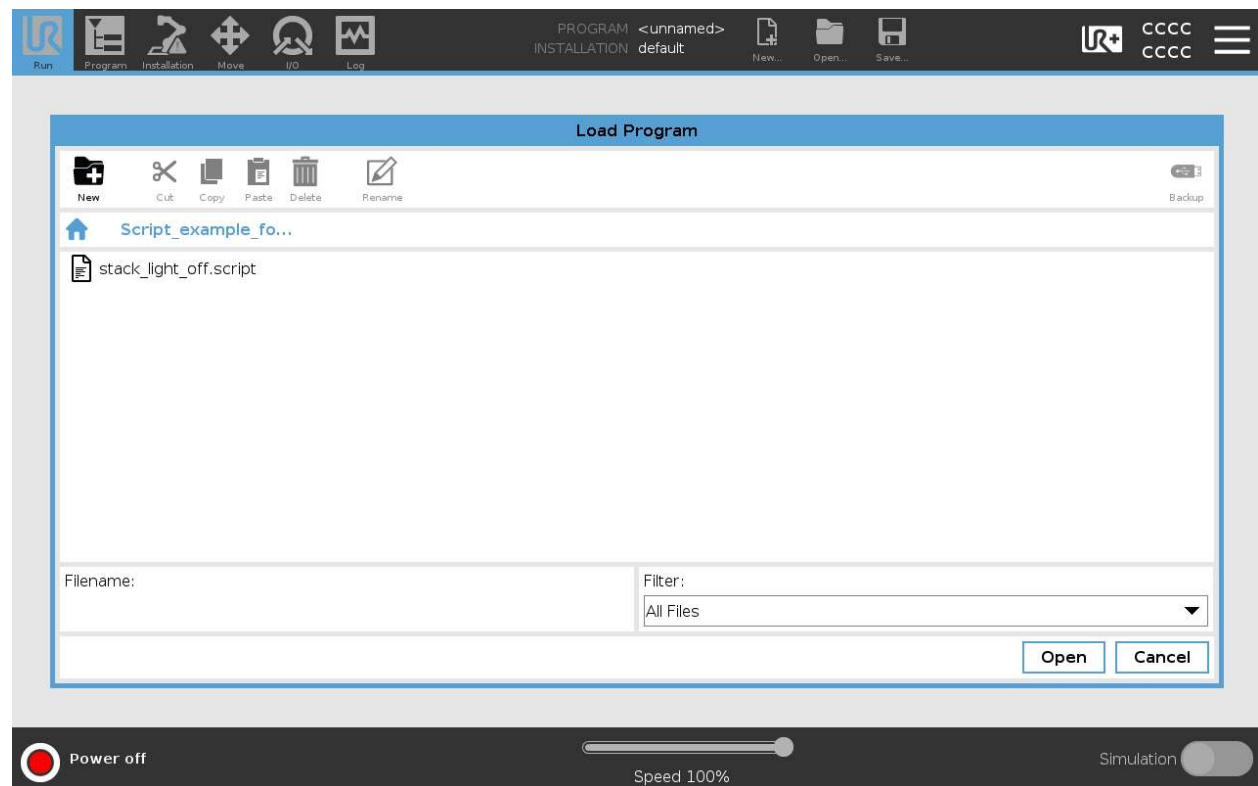


Figure 16: The .script File in the Polyscope File Navigator

Now that the .script file has been pasted onto the robot, it needs to be inserted into the program. For this example, it will be loaded into the before start sequence. To do this, in PolyScope, begin by inserting a script node into the program from the Advanced program node list. Select the File option from the drop-down menu in the upper right. Click on the Edit option to load a new file. Click on Open in the upper left, and then select your .script file. Press Exit once it has loaded onto the screen.

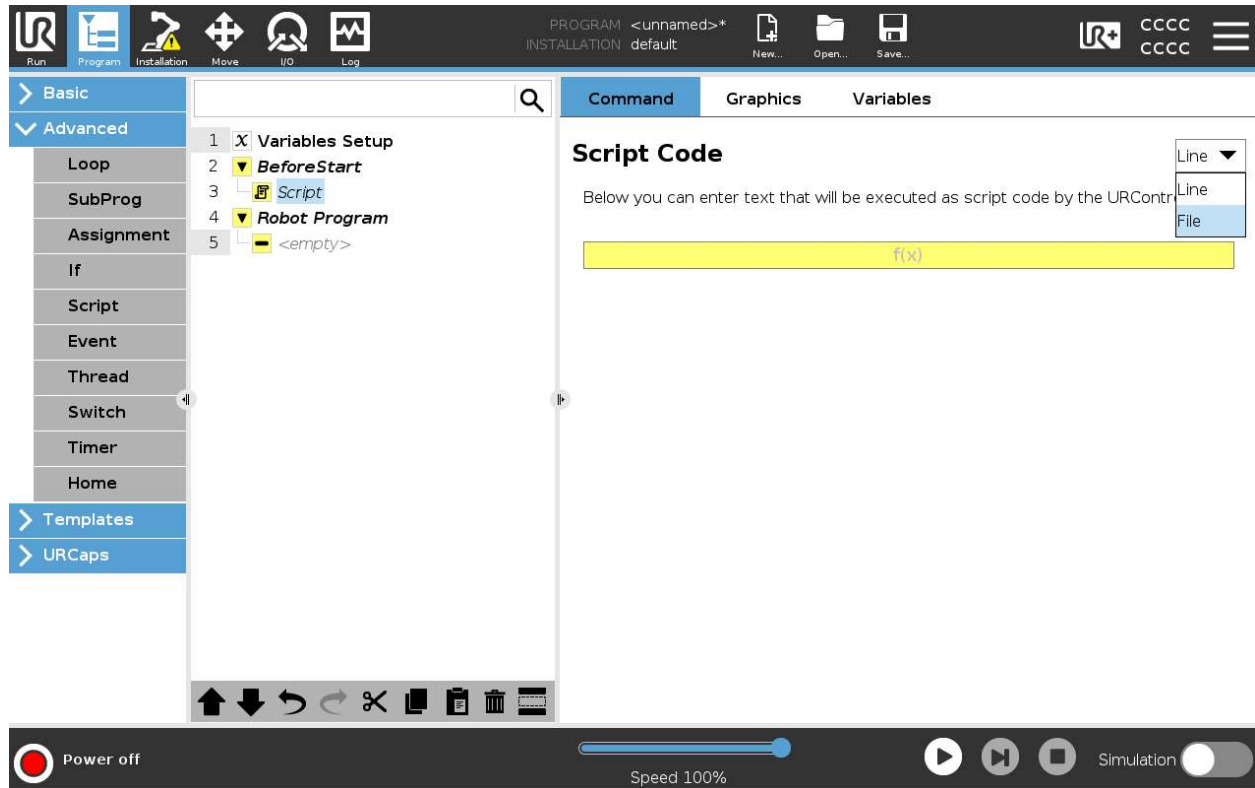


Figure 17: The File Option Location in a Script Node in PolyScope

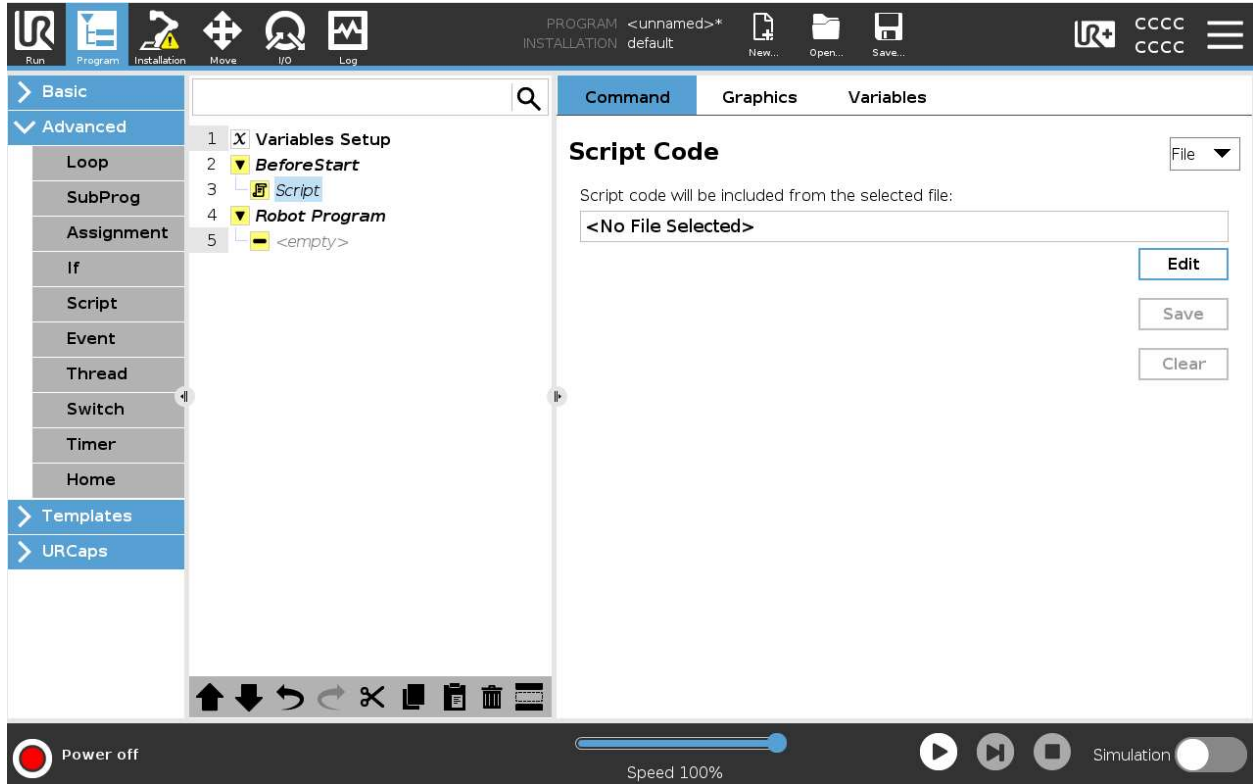


Figure 18: The Script Node with File Selected

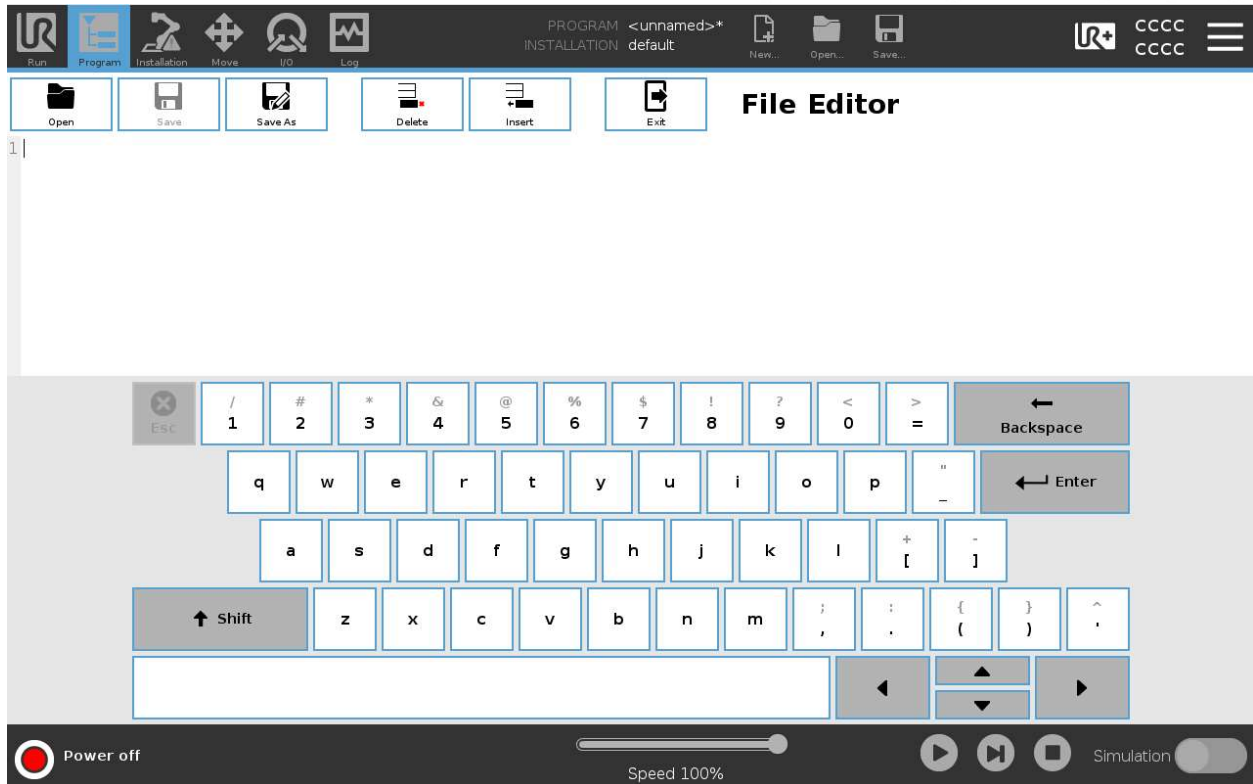


Figure 19: The Script File Editor Screen

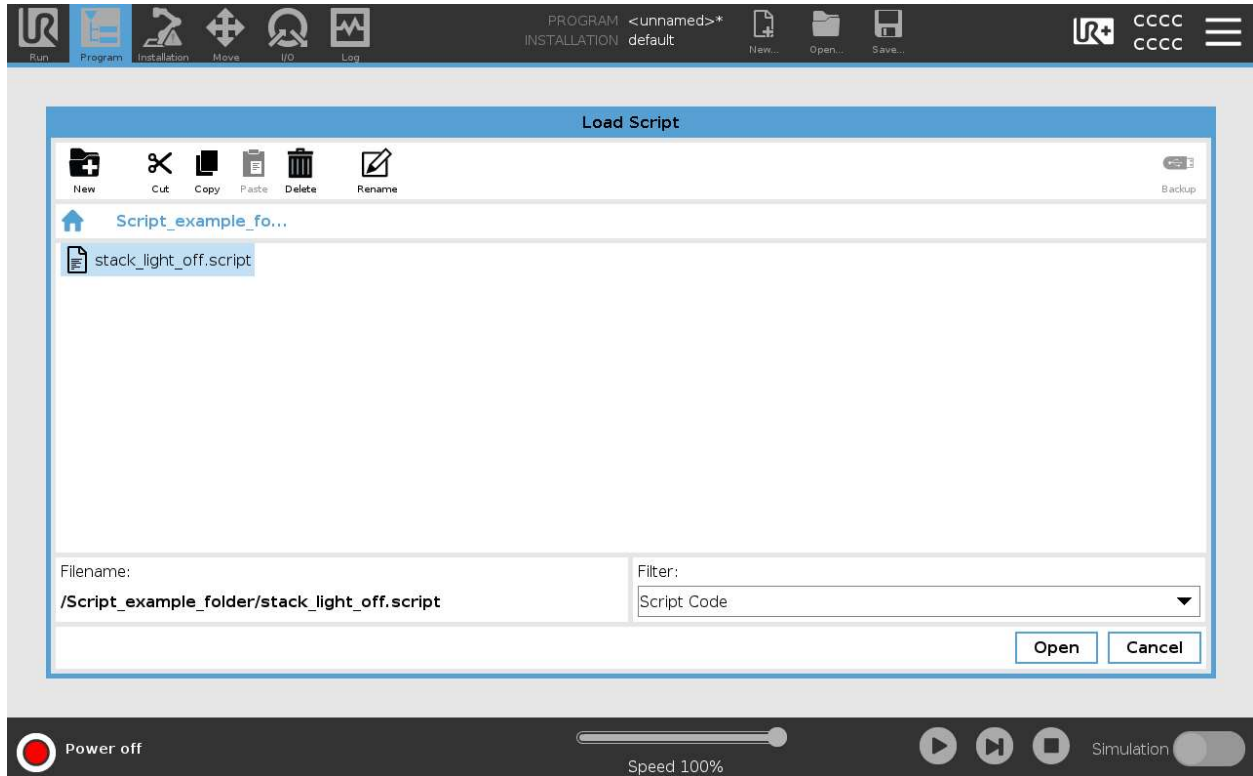


Figure 20: Select the Desired .script File

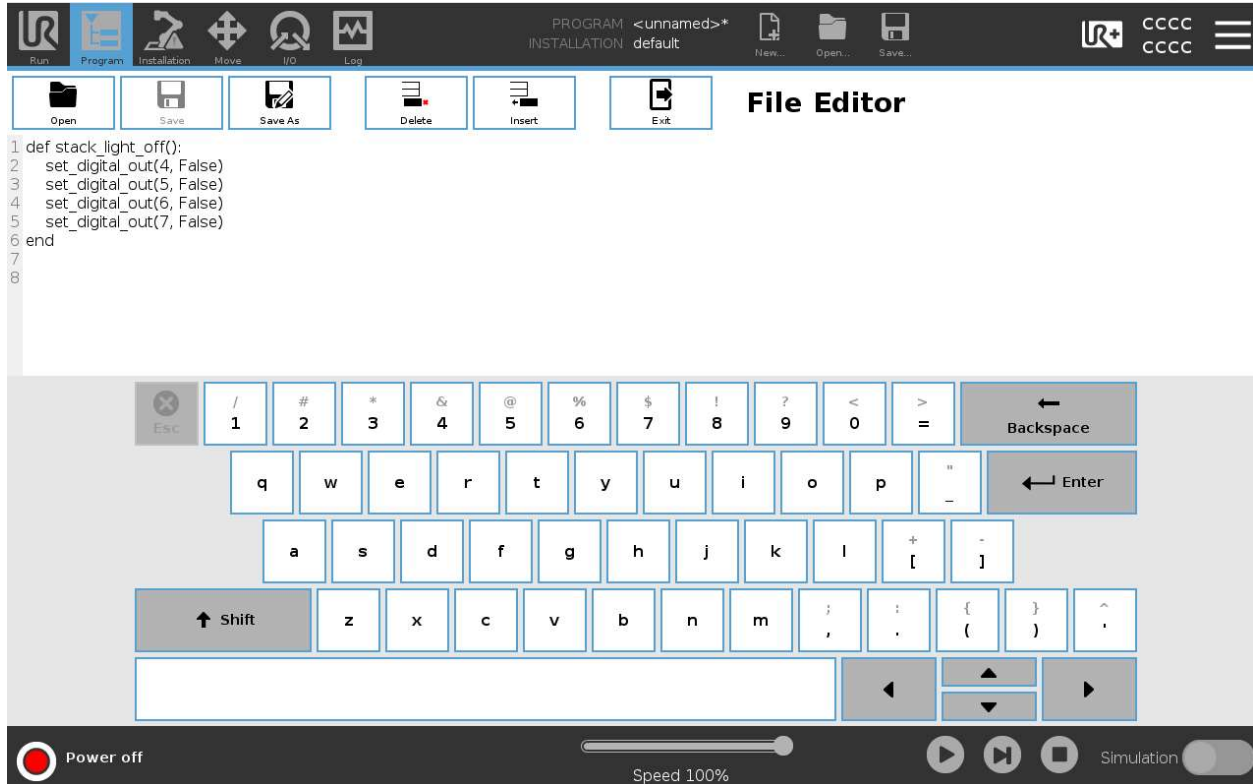


Figure 21: The .script File Loaded into the File Editor

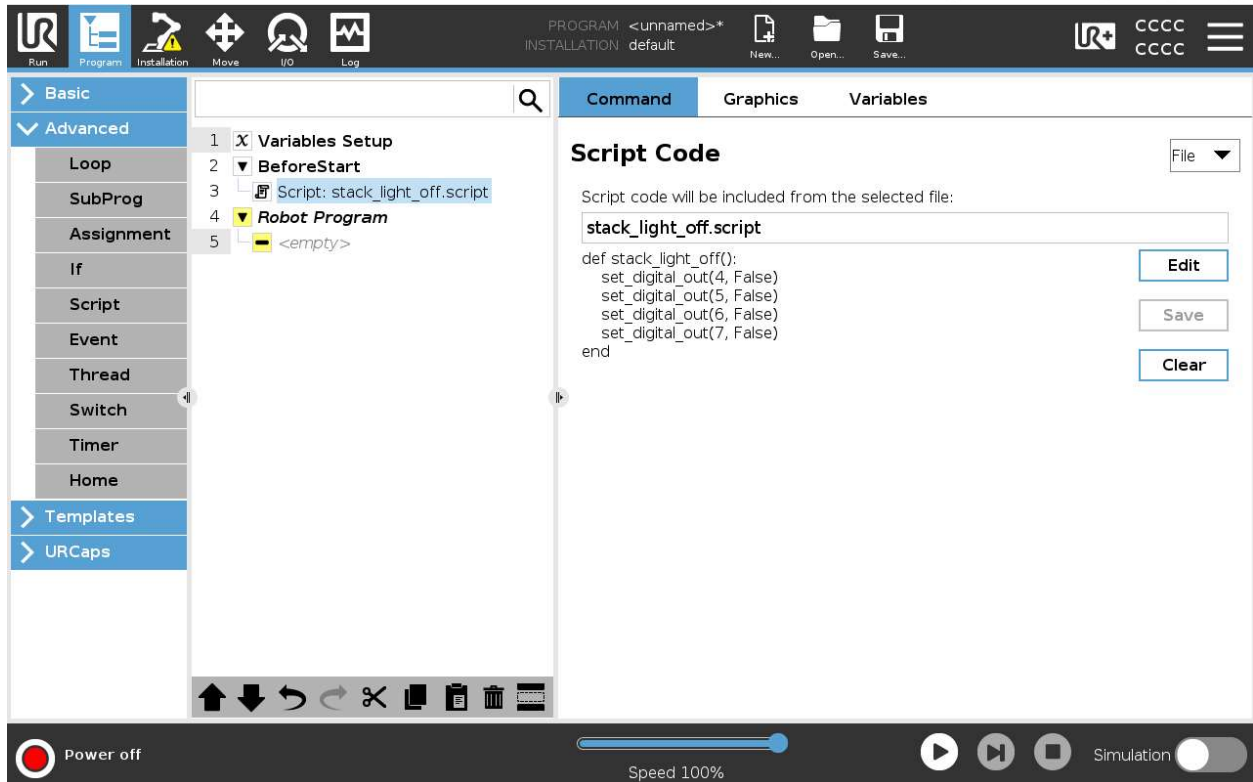


Figure 22: The .script File that has been Successfully Inserted into PolyScope

Now that the .script file has been inserted into the before start sequence, it is ready to be called. To do so, at the desired point in your code, insert a script line node, and type out the name of user defined function. Use the parentheses, and input arguments if appropriate (they are not used in this example), but do not use the colon.

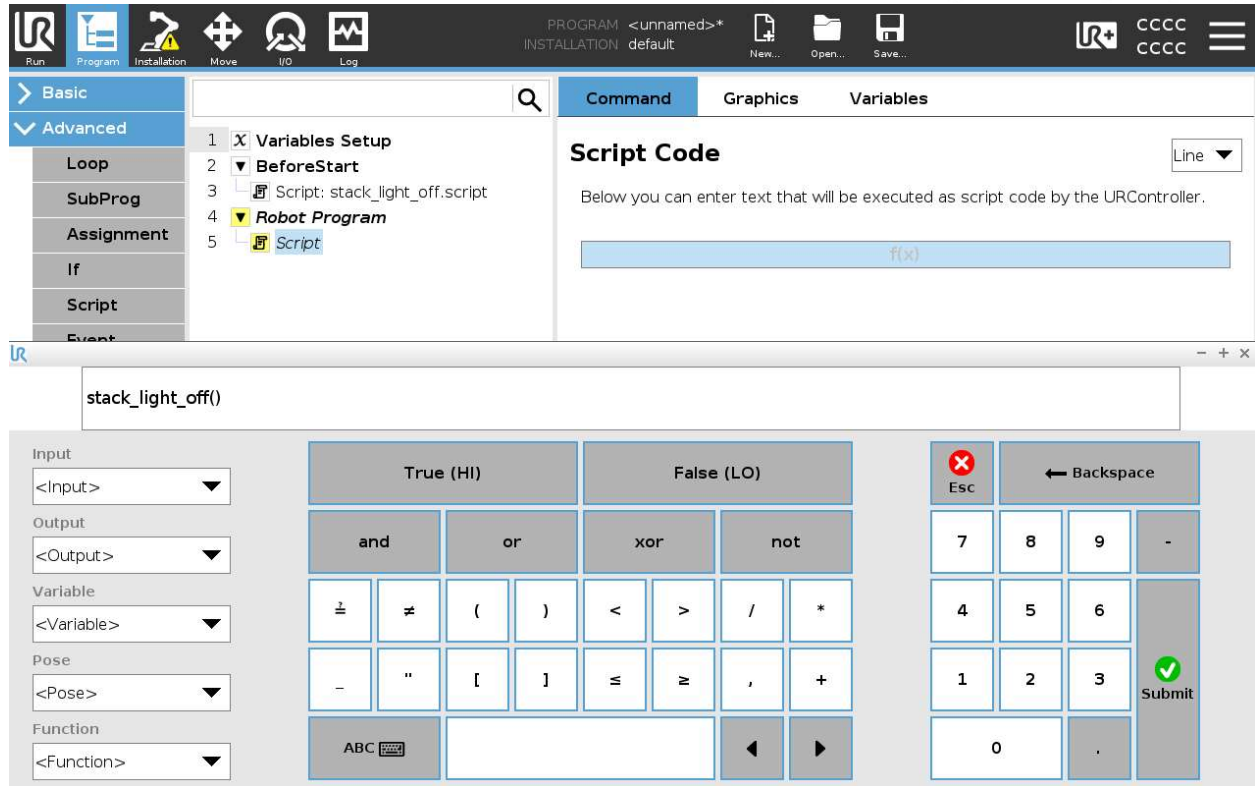


Figure 23: Calling the User Defined Function

Congratulations! You have written a user defined function in a text editor, transferred it to your robot, inserted the function definition, and called the function in the program!

9 FUNCTIONS THAT CALL OTHER FUNCTIONS

The next step in the example is a set of script functions that will be used to control a stack light. In this example, each output controls a selected color. The user will call script functions that turn off all the lights and turn on the desired color. All of these script functions will be written in one file.

To set the stack light to Green, a function will be defined named *stack_light_green()*. This function will call the previous user defined function *stack_light_off()*, and then the manual defined function *set_digital_out()*. This process is then repeated for each color.

```
1  # Use the # symbol for comments
2  # digital out 4 is the green light
3  # digital out 5 is the blue light
4  # digital out 6 is the yellow light
5  # digital out 7 is the red light
6  def stack_light_off():
7      set_digital_out(4, False)
8      set_digital_out(5, False)
9      set_digital_out(6, False)
10     set_digital_out(7, False)
11 end
12 def stack_light_green():
13     stack_light_off()
14     set_digital_out(4, True)
15 end
16 def stack_light_blue():
17     stack_light_off()
18     set_digital_out(5, True)
19 end
20 def stack_light_yellow():
21     stack_light_off()
22     set_digital_out(6, True)
23 end
24 def stack_light_red():
25     stack_light_off()
26     set_digital_out(7, True)
27 end
```

Figure 24: The File Containing all the User Defined Functions to Operate the Stack Light

This file is inserted into PolyScope the same was as before.

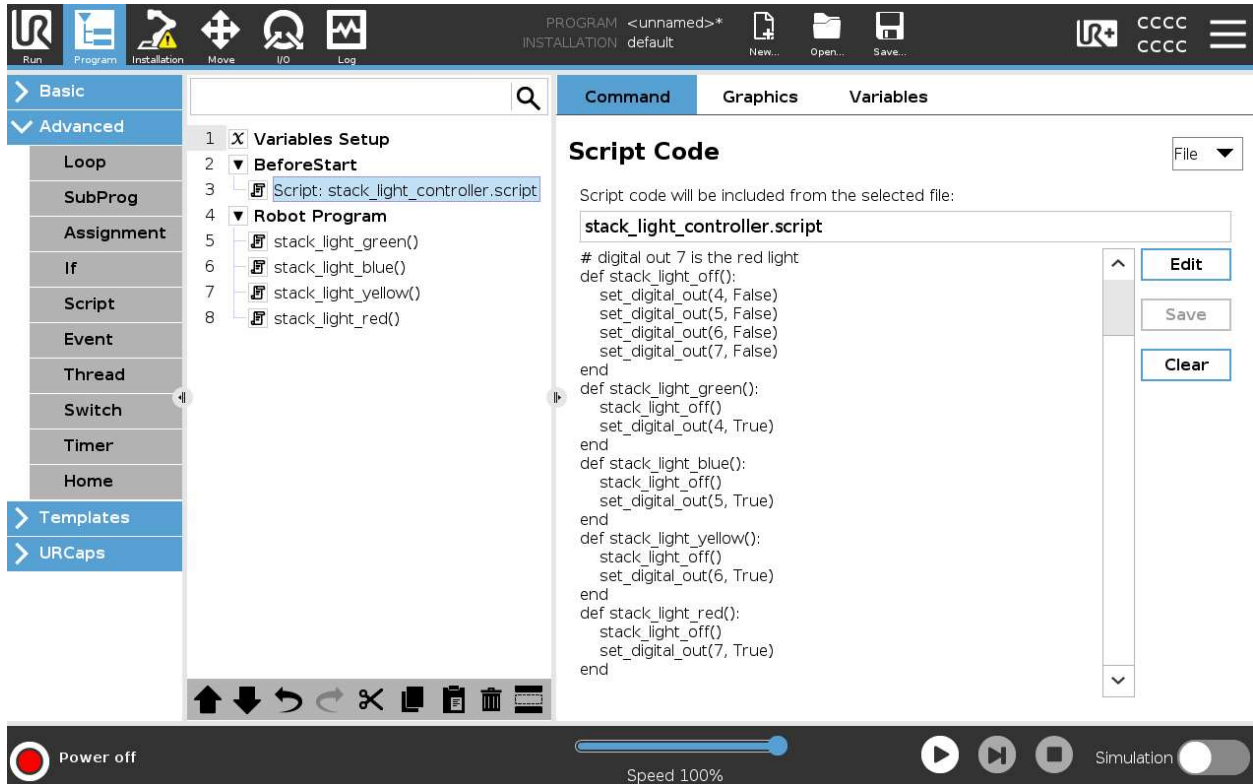


Figure 25: The Stack Light Controller File Inserted into PolyScope and its contained Functions Called

10TCP/IP SOCKET COMMUNICATION EXAMPLE

A common use of script functions is Socket TCP/IP communications. Individually, each command is a script function. They can be tedious to replicate and use frequently, especially if several commands must be used as a group. An example of this grouping is the series of commands needed to trigger a Cognex barcode camera. The camera needs to receive the specific text ">TRIGGER ON" (without the quotations), and then a carriage return and line feed command to execute the text command. After a short time, the camera again needs to receive the same command set, but with OFF instead of ON. Carriage Return and Line Feed are ASCII bytes 13 and 10, respectively. These 7 lines of code can be replaced with a single function, as shown below:

```
176 # this function turns on the top camera for a default time of 1 second
177 def top_scan(t=1.0):
178     local sleep_time=t
179     socket_send_string(">TRIGGER ON", "TopCamSocket")
180     socket_send_byte(13, "TopCamSocket")
181     socket_send_byte(10, "TopCamSocket")
182     sleep(sleep_time)
183     socket_send_string(">TRIGGER OFF", "TopCamSocket")
184     socket_send_byte(13, "TopCamSocket")
185     socket_send_byte(10, "TopCamSocket")
186     sync()
187 end
```

Figure 26: A User Defined Function Called `top_scan()` That Has a Default Scan Time of 1 Second

As a similar example, let's consider opening a socket. Opening a socket is a single line of script code, if everything works as intended. In reality, auxiliary devices may need multiple connection attempts if they are slow to boot, and your program may need specific behavior if the sockets fail to open. A comprehensive socket connection function, like the one shown below, can handle retries and failure behavior in a neat package. This function can be used to open each of your sockets, and be used in all of your programs.



```
191 # open a single camera with retries
192 # arguments ip_address and SocName need to be strings
193 def myCamOpenFunction(ip_address, SocName):
194     # function variables
195     local socketOpenLoopVar = False    # loop control variable
196     local socketRetryThreshold = 5     # retry threshold
197     global CamSocketInit = False      # establish if the socket was opened
198     local open_attempt_num = 0        # loop counter
199
200     # open the socket to the camera, with retries
201     while(socketOpenLoopVar == False):
202         CamSocketInit = socket_open(ip_address, 23, SocName)
203         if (CamSocketInit==False): # if open was unsuccessful, get ready to try again
204             socketOpenLoopVar=False    # confirm that opening was unsuccessful
205             open_attempt_num = open_attempt_num +1    # increment attempt number
206         else:
207             socketOpenLoopVar=True     # exit the loop if connection is true
208         end
209         if (open_attempt_num>socketRetryThreshold): # check against retry threshold
210             socketOpenLoopVar=True     # exit the loop if number of attempts exceeds threshold
211         end
212         sync()    # prevent runtime errors
213     end
214
215     # check for unsuccessful connections and do popups if needed
216     if (CamSocketInit==False):
217         popup("Could not connect to camera ", "Message Title", False, False, blocking=True)
218     end
219
220     return CamSocketInit    # return the true/false result of opening the socket
221 end
```

Figure 27: A User Defined Function Called myCamOpenFunction() That Has a Retry Manager and a Connection Failure Manager Built In

11 GLOBAL VS LOCAL VARIABLES

An important concept in making script libraries is the scoping of variables. Scoping refers to the domain of existence and validity of variables, or rather, where they can be used. In a computer program with libraries and references and dependencies, it's easy to unknowingly use a variable name that is already in the program somewhere else. Variable names are typically chosen for legibility to the author and future users, and often multiple authors are unaware of the variables selected by each other. Common variable names like "x," "index," "user_input," and "loop_count" are likely legitimate and intuitive selections in multiple separate instances. However, without proper scope control, it's possible to inadvertently overwrite the same variable in its other use instances.

Global variables are accessible from anywhere in the robot program, encompassing the main program file as well as any script functions that have been defined and added to the program. Local variables, by contrast, exist at their originating level. The same variable name can be used as a local variable in a function, and as a global variable in the layer above. These two instances of the variable are separate entities, despite sharing a name. This prevents a user from accidentally overwriting a variable that is used elsewhere in a library.

It is best practice to make all variables in supporting libraries local variables, particularly if the program author is not the library author. The exception is variables that may be useful or necessary for the program author to access or monitor. For these global variables that originate from within a script function, it is recommended to use a clear and consistent naming convention that the program author would not naturally select. An example naming convention for global variables created within a script function could be: "COMPANYNAME_LIBRARYNAME_VARIABLENAME".

If a variable is created in a function without explicitly being declared as local or global, it will default to global.

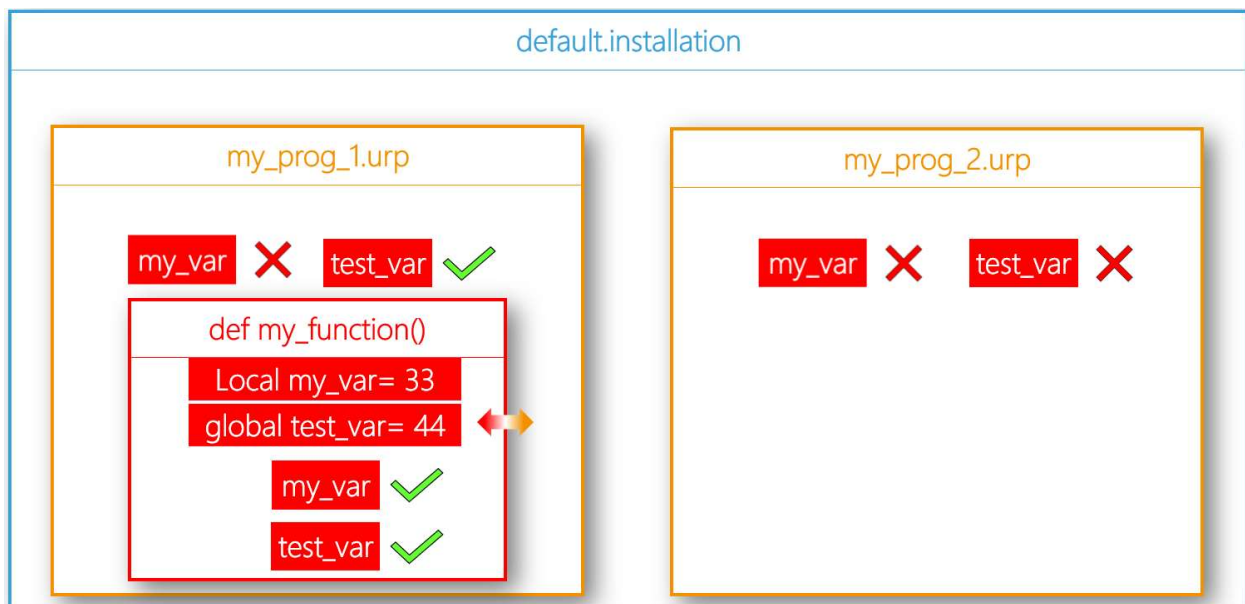


Figure 28: Scoping Diagram Illustrating Where the Example Variables `my_var` and `test_var` are Accessible



```
1  # local variable example
2  a = 4 # used for this example
3
4  # a function that multiplies by 2
5  def times2(argument1):
6      local my_var = argument1 * 2
7      return my_var
8  end
9
10 # a function that multiplies by 3
11 def times3(arguemnt2):
12     local my_var = arguemnt2 * 3
13     return my_var
14 end
15
16 def multical1(argument3):
17     local my_var = argument3
18     local answer1 = times2(my_var)
19     local answer2 = times3(my_var)
20     local a = [answer1, answer2]
21     return a
22 end
23
24 # use the functions
25 c = multical1(a)
26
27 # display the value of the variable, should be [8, 12]
28 popup(to_str(c), title = "The value of your variable is: " )
```

Figure 29: Example Code Where the Variable my_var is Local to Several Functions

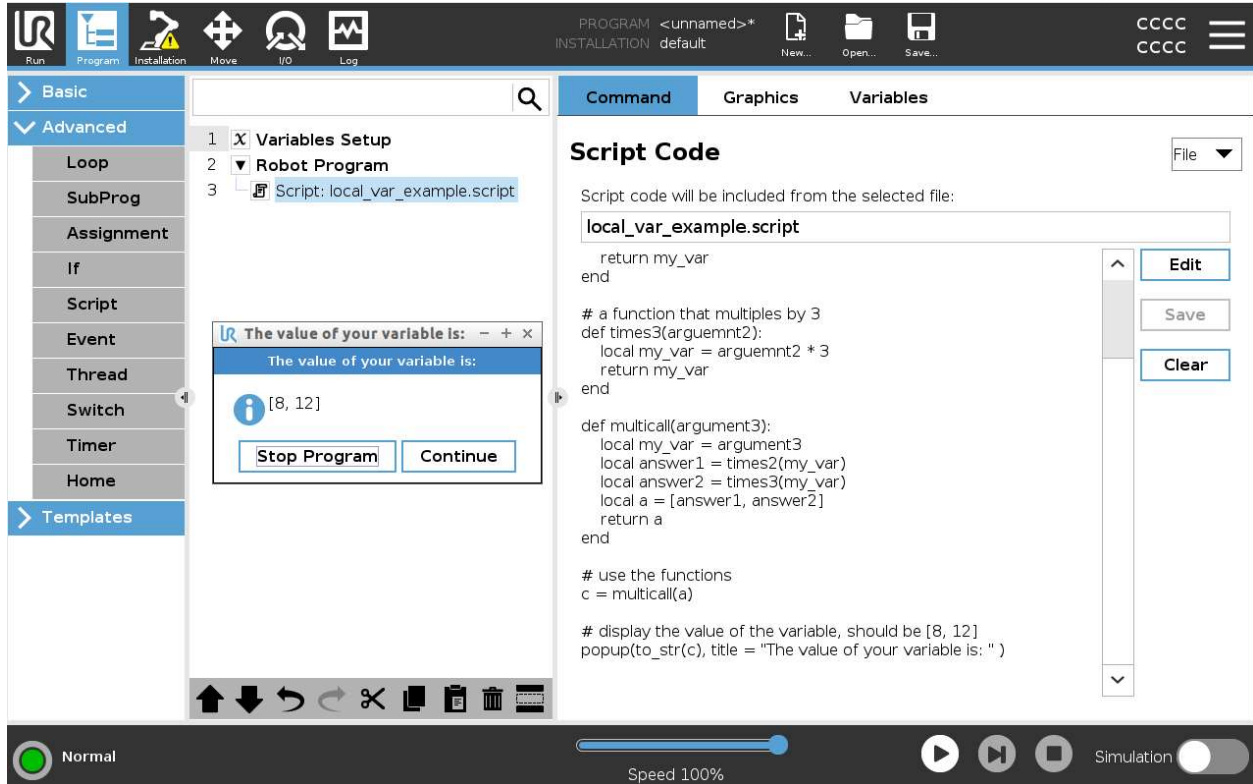


Figure 30: The Example Code from the Previous Image, Running in PolyScope

12 WRITING AN ENTIRE PROGRAM IN URSRIPT

You can write an entire robot program in URScript code. This is rare and typically unadvised, as PolyScope is exceptionally user friendly for handling most things a robot will do. However, as a conceptual exercise, it is interesting to see. The program executes and runs, moving the robot to waypoints and updating the stack light at each waypoint. While this program could certainly be done with PolyScope program nodes, it illustrates what can be done with URScript.

```
1  # put this script fine in the before start sequence
2  # this has 3 sections: helper functions, before start, and define the main program
3  # this file will be inserted into the before start sequence
4  # the before start action should auto-run
5  # in polyscope, the main program will be called just one function: myMainProgram()
6  # the waypoints in this program were selected for a UR5e
7
8  # add your helper functions here
9  #####
10 # stack light controller helper functions
11 # digital out 4 is the green light
12 # digital out 5 is the blue light
13 # digital out 6 is the yellow light
14 # digital out 7 is the red light
15 def stack_light_off():
16     set_digital_out(4, False)
17     set_digital_out(5, False)
18     set_digital_out(6, False)
19     set_digital_out(7, False)
20 end
21 def stack_light_green():
22     stack_light_off()
23     set_digital_out(4, True)
24 end
25 def stack_light_blue():
26     stack_light_off()
27     set_digital_out(5, True)
28 end
29 def stack_light_yellow():
30     stack_light_off()
31     set_digital_out(6, True)
32 end
33 def stack_light_red():
34     stack_light_off()
35     set_digital_out(7, True)
36 end
```

Figure 31: The First Half of the URScript File that Runs an Entire Robot Program



```
37
38 # put your before start actions here
39 #####
40 # define some waypoints
41 pt1 = p[-0.1, -0.5, 0.25, 0, 3.14, 0] # point 1 in waypoint form
42 pt2 = p[0.15, -0.2, 0.3, 0, 3.14, 0] # point 2 in waypoint form
43 pt3 = pose_add(pt1, p[0,0,0.2,0,0,0]) # point 3, 200mm above point 1
44 pt4 = pose_add(pt2, p[0,0,0.15,0,0,0]) # point 4, 150mm above point 2
45
46 # set my I/O to start
47 stack_light_yellow()
48
49 # move up, then to the first position
50 init_pose = get_actual_tcp_pose() # find current position
51 retreat_pose = pose_add(init_pose, p[0,0,0.1,0,0,0]) # find a position 100mm up
52 movel(retreat_pose, a=1, v=0.1) # move to the retreat pose
53 movej(pt1) # move to point 1
54
55 # define your main program here
56 #####
57 def myMainProgram():
58     stack_light_green()
59     movel(pt2)
60     stack_light_blue()
61     movej(pt3)
62     stack_light_yellow()
63     movel(pt4)
64     stack_light_red()
65     movej(pt1)
66 end
```

Figure 32: The Second Half of the URScript File that Runs an Entire Robot Program

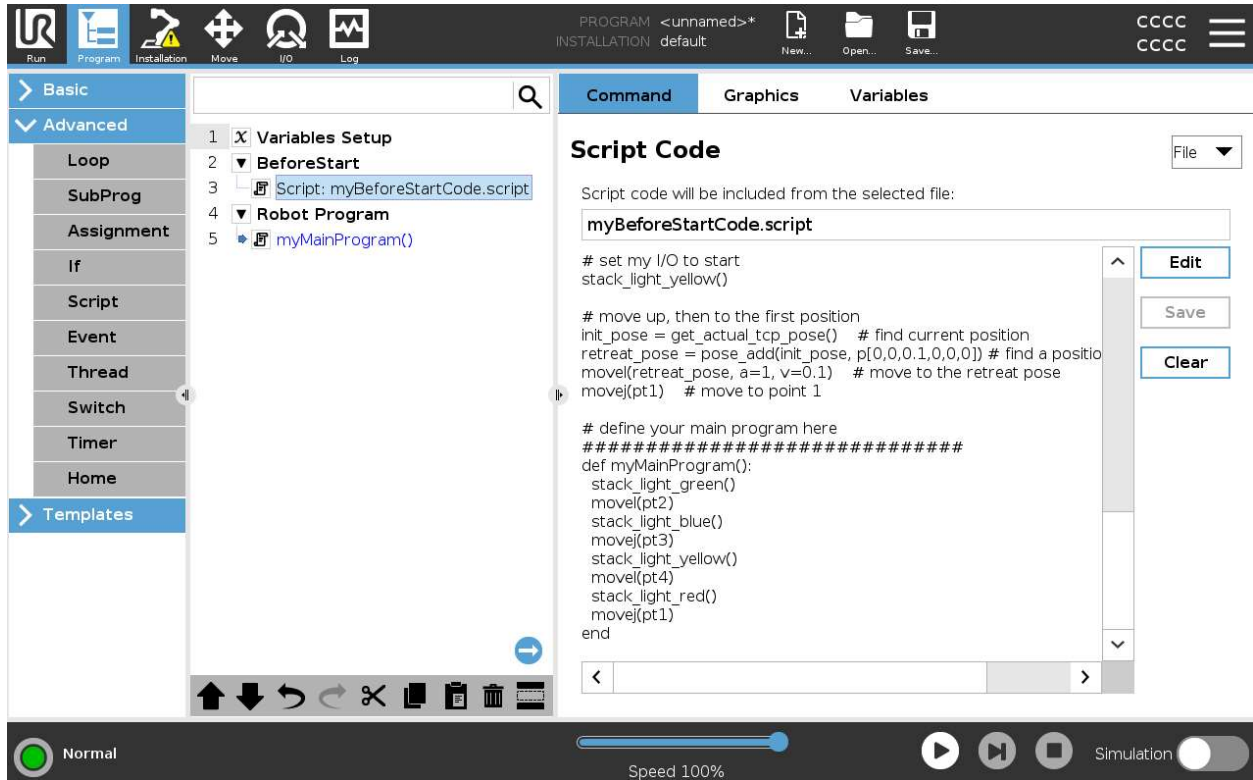


Figure 33: The PolyScope Program that Runs from Two Script Node: One to Define Everything, and One to Call the Main Program.



13 ORGANIZING AND MANAGING YOUR LIBRARY

Once you have made a few functions, it is important to keep track of them. Treat your code as a revision-controlled engineering document, with authors, version numbers, and change logs. If a function has changed to the point that a call to the original function wouldn't work on the current version, you now have a new non-interchangeable function that must use a different name.

```
1  # Stack Light Controller Function
2  # Version: 1.1
3  # Date: 10/31/2023
4  # Author: Phillip Grambo
5  # Contact: pgra@universal-robots.com
6  # Comments: added the red stack light to D07
7
8  # Version: 1.0
9  # Date: 5/23/2023
10 # Author: Phillip Grambo
11 # Contact: pgra@universal-robots.com
12 # Comments: initial version
13
14 # digital out 4 is the green light
15 # digital out 5 is the blue light
16 # digital out 6 is the yellow light
17 # digital out 7 is the red light
18 def stack_light_off():
19     set_digital_out(4, False)
20     set_digital_out(5, False)
21     set_digital_out(6, False)
22     set_digital_out(7, False)
23 end
24 def stack_light_green():
25     stack_light_off()
26     set_digital_out(4, True)
27 end
28 def stack_light_blue():
29     stack_light_off()
30     set_digital_out(5, True)
31 end
32 def stack_light_yellow():
33     stack_light_off()
34     set_digital_out(6, True)
35 end
36 def stack_light_red():
37     stack_light_off()
38     set_digital_out(7, True)
39 end
```

Figure 34: A URScript Function with Revision Control Comments

As your libraries expand, consider using repositories with check-in/check-out and restore-to-previous-version capabilities. GitHub is a popular option for this.

To take backups from a robot, consider using our Magic File for backups (<https://www.universal-robots.com/download/software-e-series/support/magic-files/magic-file-backup-all-programs/>) for convenience. Plug it into your robot, and it will create a zip folder with all your programs on the flash drive. The name of the zip folder will have the serial number of the robot, as well as the date and time stamp of the backup. Keep these backups, and have read-only master copies.

Part of managing a library is testing and verification of the functions. It should be noted that some errors are only found at runtime when the functions are called upon. Some errors, like a missing “end” statement to wrap a user-defined function, will be caught immediately when the program is played. However, if you accidentally spell a function name wrong or use the wrong variable type (ex: attempt to `move()` to a list instead of a pose), these errors will not be identified until program execution reaches that point. As such, it is best practice to call your functions in a test environment to ensure functionality, and not merely confirm that PolyScope can compile the resultant file.

14CHANGE LOG

Version	Date	Author	Contact	Comments
1.0	1/29/2024	PGRA	pgra@universal-robots.com	Initial release